Speech and Language Processing, selected pages for class 1

Daniel Jurafsky and James H. Martin, Speech and Language Processing, 3rd edition draft, August 2026. The whole draft is free at https://web.stanford.edu/~jurafsky/slp3/ and the authors allow its use in classes. These pages are for CSE 4811 / DS 4213 Natural Language Processing, Fall 2026, Department of Computer Science and Engineering, United International University. Page numbers are the book's.

- Section 2.1, Words, pages 34 to 36
- Section 2.4, Subword tokenization: byte-pair encoding, and section 2.5, Corpora, pages 42 to 47
- Section 5.3, Simple count-based embeddings, and section 5.4, Cosine for measuring similarity, pages 132 to 135

2.1 Words

How many words are in the following sentence?

```
They picnicked by the pool, then lay back on the grass and
looked at the stars.
```

This sentence has 16 words if we don't count punctuation as words, 18 if we count punctuation. Whether we treat period (“.”), comma (“,”), and so on as words depends on the task. Punctuation is critical for finding boundaries of things (commas, periods, colons) and for identifying some aspects of meaning (question marks, exclamation marks, quotation marks). Large language models generally count punctuation as separate words.

Spoken language introduces other complications with regard to defining words. What about this utterance from a spoken conversation? (**Utterance** is the technical linguistic term for the spoken correlate of a sentence).

utterance

```
I do uh main- mainly business data processing
```

disfluency
fragment
filled pause

This utterance has two kinds of **disfluencies**. The broken-off word *main-* is called a **fragment**. Words like *uh* and *um* are called **fillers** or **filled pauses**. Should we consider these to be words? Again, it depends on the application. If we are building a speech transcription system, we might want to eventually strip out the disfluencies. But we also sometimes keep disfluencies around. Disfluencies like *uh* or *um* are actually helpful in speech recognition in predicting the upcoming word, because they may signal that the speaker is restarting the clause or idea, and so for speech recognition they are treated as regular words. Because different people use different disfluencies they can also be a cue to speaker identification. In fact Clark and Fox Tree (2002) showed that *uh* and *um* have different meanings in English. What do you think they are?

Perhaps most important, in thinking about what is a word, we need to distinguish two ways of talking about words that will be useful throughout the book. Let's consider a sentence. If we ignore punctuation, this sentence has 16 words, but only 14 unique words (because *the* occurs 3 times).

```
They picnicked by the pool, then lay back on the grass and
looked at the stars.
```

word type

We call the set of unique words in a corpus the word **types**. So the sentence above has 14 word types, and if the vocabulary of a whole language (the set of possible words in the language) is V, the number of types is the **vocabulary size** $|V|$. We call

word instance

word **instances** the number N of running words, so 16 in the example above.[1]

We still have decisions to make! For example, should we consider a capitalized string (like *They*) and one that is uncapitalized (like *they*) to be the same word **type**? The answer is that it depends on the task! *They* and *they* might be lumped together as the same type in some tasks where we care less about the formatting, while for other tasks, capitalization is a useful feature and is retained. Sometimes we keep around two versions of a particular NLP model, one with capitalization and one without capitalization.

So far we have been talking about **orthographic words**: words based on our English writing system. But there are many other possible ways to define words.

[1] In earlier tradition, and occasionally still, you might see word instances referred to as word *tokens*, but we now try to reserve the word *token* instead to mean the output of subword tokenization algorithms.

Corpus	Types = $\|V\|$	Instances = N
Shakespeare	29 thousand	884 thousand
Brown corpus	38 thousand	1 million
Switchboard telephone conversations	20 thousand	2.4 million
COCA	2 million	440 million
Google n-grams	13 million	1 trillion

Figure 2.1 Rough numbers of wordform types and instances for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

For example, while orthographically `I'm` is one word, grammatically it functions as two words: the subject pronoun `I` and the verb `'m`, short for `am`.

The distinctions get even harder to make once we start to think about other languages. For example the writing systems of languages like Chinese, Japanese, and Thai simply don't have orthographic words at all! That is, they don't use spaces to mark potential word-boundaries. In Chinese, for example, words are composed of characters (called **hanzi** in Chinese). Each character generally represents a single unit of meaning (called a **morpheme**, introduced below) and is pronounceable as a single syllable. Words are roughly between 1.5 and 1.9 characters long on average, depending on how you measure. But since Chinese has no orthographic words, deciding what counts as a word in Chinese is complex. For example, consider the following sentence:

hanzi

(2.1) 姚明进入总决赛 yáo míng jìn rù zǒng jué sài
"Yao Ming reaches the finals"

As Chen et al. (2017b) point out, this could be treated as 3 words (a definition of words called the 'Chinese Treebank' definition, in which Chinese names (family name followed by personal names) are treated as a single word):

(2.2) 姚明 进入 总决赛
YaoMing reaches finals

But the same sentence could be treated as 5 words ('Peking University' standard), in which names are separated into their own units and some adjectives appear as distinct words:

(2.3) 姚 明 进入 总 决赛
Yao Ming reaches overall finals

Finally, it is possible in Chinese simply to ignore words altogether and use characters as the basic elements, treating the sentence as a series of 7 characters, which works pretty well for Chinese since characters are at a reasonable semantic level for most applications (Li et al., 2019):

(2.4) 姚 明 进 入 总 决 赛
Yao Ming enter enter overall decision game

But that method doesn't work for Japanese and Thai, where the individual character is too small a unit.

These issues with defining words make it hard to use words as the basis for tokenizing text in NLP across languages.

But there's another problem with words. There are too many of them!!! How many words are there in English? When we speak about the number of words in the language, we are generally referring to word types. Fig. 2.1 shows the rough numbers of types and instances computed from some English corpora.

You will notice that the larger the corpora we look at, the more word types we find! That suggests that there is not a clear answer to how many words there are; the answer keeps growing as we see more data! We can see this fact mathematically because the relationship between the number of types $|V|$ and number of instances N is called **Herdan's Law** (Herdan, 1960) or **Heaps' Law** (Heaps, 1978) after its discoverers (in linguistics and information retrieval respectively). It is shown in Eq. 2.5, where k and β are positive constants, and $0 < \beta < 1$.

Herdan's Law
Heaps' Law

$$|V| = kN^{\beta} \tag{2.5}$$

The value of β depends on the corpus size and the genre; numbers from 0.44 to 0.56 or even higher have often been reported. Roughly we can say that the vocabulary size for a text goes up a little faster than the square root of its length in words.

There are also variants of the law, which capture the fact that we can distinguish roughly two classes of words. One is **function words**, the grammatical words like English *a* and *of*, that tend not to grow indefinitely (a language tends to have a fixed number of these). The other is **content words**: nouns, adjectives and verbs that tend to have meanings about people and places and events. Nouns, and especially particular nouns like names and technical terms do tend to grow indefinitely. So models that are sensitive to this difference between function words and content words have one value of β for the initial part of the corpus where all words are still appearing, and then a second β afterwards for when only the content words are still appearing. Fig. 2.2 shows an example from Tria et al. (2018) showing two values of β (called γ in their figure) for Heaps law computed on the Gutenberg corpus of books.

function words
content words

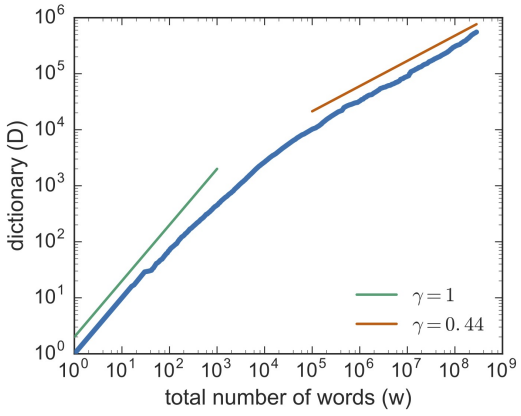


Figure 2.2 The thick blue line shows Vocabulary size $|V|$ (called D in their figure) as a function of text length (their w), computed on the Gutenberg corpus of publicly available books. Note that at the beginning of the corpus, we see both common and function words and the relationship between corpus size and vocabulary is roughly linear (green line, $\gamma = 1$). Later, after the function words have mainly appeared, the number of new words slows down and is closer to the square root of the corpus size. Figure from Tria et al. (2018).

The fact that words grow without end leads to a problem for any computational model. No matter how big our vocabulary, we will never have a vocabulary that captures all the possible words that might occur! That means that our computational model will constantly see **unknown words**: words that it has never seen before. This is a huge problem for machine learning models.

Middle Eastern, and African scripts map to two bytes, most Chinese, Japanese, and Korean characters map to three bytes, and rarer CJKV characters and emojis and some symbols map to 4 bytes.

UTF-8 has a number of advantages. It's relatively efficient, using fewer bytes for commonly-encountered characters, it doesn't use zero bytes (except when literally representing the NULL character which is U+0000), it's backwards compatible with ASCII, and it's self-synchronizing, meaning that if a file is corrupted, it's always possible to find the start of the next or prior character just by moving up to 3 bytes left or right.

Unicode and Python: Starting with Python 3, all Python strings are stored internally as Unicode, each string a sequence of Unicode code points. Thus string functions and regular expressions all apply natively to code points. For example, functions like `len()` of a string return its length in characters, i.e., code points, not its length in bytes.

When reading or writing from a file, however, the code points need to be encoded and decoded using a method like UTF-8. That is, every file is encoded in some encoding. If it's not UTF-8, it's an older encoding method like ASCII or Latin-1 (iso_8859_1). There is no such thing as a text file without an encoding. The encoding method is specified in Python when opening a file for reading and writing.

2.4 Subword Tokenization: Byte-Pair Encoding

tokenization
tokens

Tokenization, the first stage of natural language processing, is the process of segmenting the running input text into **tokens**.

We've seen three candidates for tokens: words, morphemes and characters. But each has problems as a unit. Words and morphemes seem approximately at the right level for NLP processing, since they tend to have consistent meanings, but they are challenging to define formally. Characters are clearer to define, but seem too small a unit to choose for tokens.

In this section we introduce what we do in practice for NLP: use a data-driven approach to define tokens that will generally result in units about the size of morphemes or words, but occasionally use units as small as characters.

Why tokenize the input? One reason is that converting an input to a deterministic fixed set of units means that different algorithms and systems can agree on simple questions. For example, How long is this text? (How many units are in it?). Or: Is *don't* or *New York* one token or two? Standardizing is thus essential for replicability in NLP experiments, and many algorithms that we introduce in this book (like the **perplexity** metric for language models) assume that all texts have a fixed tokenization.

Tokenization algorithms that include smaller tokens for morphemes and letters also eliminate the problem of **unknown words**. What are these? As we will see in the next chapter, NLP algorithms often learn some facts about language from one corpus (a **training** corpus) and then use these facts to make decisions about a separate **test** corpus and its language. Thus if our training corpus contains, say the words *low*, *new*, and *newer*, but not *lower*, then if the word *lower* appears in our test corpus, our system will not know what to do with it.

subwords

To deal with this unknown word problem, modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called **subwords**. Sub-

words can be arbitrary substrings, or they can be meaning-bearing units like the morphemes *-est* or *-er*. In modern tokenization schemes, many tokens are words, but other tokens are frequently occurring morphemes or other subwords like *-er*. Every unseen word can thus be represented by some sequence of known subword units. For example, if we had happened not to ever see the word *lower*, when it appears we could segment it successfully into *low* and *er* which we had already seen. In the worst case, a really unusual word (perhaps an acronym like *GRPO*) could be tokenized as a sequence of individual letters if necessary.

Two tokenization algorithms are widely used in modern language models: **byte-pair encoding** (BPE) (Sennrich et al., 2016), and **unigram language modeling** (ULM) (Kudo, 2018).[2] In this section we introduce the **byte-pair encoding** or **BPE** algorithm (Sennrich et al., 2016; Gage, 1994); see Fig. 2.6.

BPE

Like most tokenization schemes, the BPE algorithm has two parts: a **training phase**, and the **encoder** phase. In general in the token training phase the tokenizer takes a raw training corpus (usually roughly pre-separated into words, for example by whitespace) and induces a vocabulary, a set of tokens. Then in the encoding phase the tokenizer takes a raw test sentence and encodes it into the tokens in the vocabulary that were learned in training.

2.4.1 BPE training

The **BPE** training algorithm iteratively merges frequent neighboring tokens to create longer and longer tokens. The algorithm begins with a vocabulary that is just the set of all individual characters. It then examines the training corpus, and finds the two characters that are most frequently adjacent. Imagine our original corpus is 10 characters long, using a vocabulary of 5 characters, {A, B, C, D, E}:

```
A B D C A B E C A B
```

The most frequent neighboring pair of characters is "A B" so we merge those, add a new merged token 'AB' to the vocabulary, and replace every adjacent 'A' 'B' in the corpus with the new 'AB':

```
AB D C AB E C AB
```

Now we have a vocabulary of 6 possible tokens {A, B, C, D, E, AB}, and the corpus has length 7. And now the most frequent pair of tokens is "C AB", so we merge those, leading to a vocabulary with 7 tokens {A, B, C, D, E, AB, CAB}, and the corpus has length 5.

```
AB D CAB E CAB
```

The algorithm continues to count and merge, creating new longer and longer character strings, until k merges have been done creating k novel tokens; k is thus a parameter of the algorithm. The resulting vocabulary consists of the original set of characters plus k new symbols. That's the core of the algorithm.

The only additional complication is that in practice, instead of running on the raw sequence of characters, the algorithm is usually run only *inside* words. That is, the algorithm does not merge across word boundaries. To do this, the input corpus is often first separated at white space and punctuation (using the regular expressions that we define later in the chapter). This gives a starting set of strings, each corresponding to the characters of a word, (with the white space usually attached to the

[2] The **SentencePiece** library includes implementations of both of these (Kudo and Richardson, 2018), and people sometimes use the name **SentencePiece** to simply mean ULM tokenization.

start of the word), together with the counts of the words. Then while counts come from a corpus, merges are only allowed within the strings.

Let's see how the full algorithm thus works on this tiny synthetic corpus, where we've explicitly marked the spaces between words:[3]

(2.11) `set␣new␣new␣renew␣reset␣renew`

First, we'll break up the corpus into words, with leading whitespace, together with their counts; no merges will be allowed to go beyond these word boundaries. The result looks like the following list of 4 words and a starting vocabulary of 7 characters:

corpus		**vocabulary**
2	␣ n e w	␣, e, n, r, s, t, w
2	␣ r e n e w	
1	s e t	
1	␣ r e s e t	

The BPE training algorithm first counts all pairs of adjacent symbols: the most frequent is the pair `n e` because it occurs in `new` (frequency of 2) and `renew` (frequency of 2) for a total of 4 occurrences. We then merge these symbols, treating `ne` as one symbol, and count again:

corpus		**vocabulary**
2	␣ ne w	␣, e, n, r, s, t, w, ne
2	␣ r e ne w	
1	s e t	
1	␣ r e s e t	

Now the most frequent pair is `ne w` (total count=4), which we merge.

corpus		**vocabulary**
2	␣ new	␣, e, n, r, s, t, w, ne, new
2	␣ r e new	
1	s e t	
1	␣ r e s e t	

Next `␣ r` (total count of 3) gets merged to `␣r`, and then `␣r e` (total count 3) gets merged to `␣re`. The system has essentially induced that there is a word-initial prefix `re-`:

corpus		**vocabulary**
2	␣ new	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re
2	␣re new	
1	s e t	
1	␣re s e t	

If we continue, the next merges are:

merge	**current vocabulary**
(␣, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new
(␣re, new)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew
(s, e)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se
(se, t)	␣, e, n, r, s, t, w, ne, new, ␣r, ␣re, ␣new, ␣renew, se, set

[3] Yes, we realize this isn't a particularly likely or exciting sentence.

function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens k times
$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
$V \leftarrow V + t_{NEW}$ # update the vocabulary
Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Figure 2.6 The training part of the BPE algorithm for taking a corpus broken up into individual characters or bytes, and learning a vocabulary by iteratively merging tokens. Figure adapted from Bostrom and Durrett (2020).

2.4.2 BPE encoder

Once we've learned our vocabulary, the BPE **encoder** is used to tokenize a test sentence. The encoder just runs on the test data the merges we have learned from the training data. It runs them in the order we learned them (i.e., greedily, meaning starting from the most frequent in the training data). The frequencies in the test data don't play a role, just the frequencies in the training data. So first we segment each test sentence word into characters. Then we apply the first rule: replace every instance of `n e` in the test corpus with `ne`, and then the second rule: replace every instance of `ne w` in the test corpus with `new`, and so on. By the end of course many of the merges simply recreated words in the training set. But the merges also created knowledge of morphemes like the `re-` prefix (that might appear in perhaps unseen combinations like `revisit` or `rearrange`), or the morpheme `new` without an initial space (hence word-internal) that might appear at the start of sentences or in words unseen in training like `anew`.

Of course in real settings BPE is run with tens of thousands of merges on a very large input corpus, to produce vocabulary sizes of 50,000, 100,000, or even 200,000 tokens. The result is that most words can be represented as single tokens, and only the rarer words (and unknown words) will have to be represented by multiple tokens. At least for English. For multilingual systems, the tokens can be dominated by English, leaving fewer tokens for other languages, as we'll discuss below.

2.4.3 BPE in practice

The example above just showed simple BPE learning from sequences of ASCII bytes. How does BPE work with Unicode input? We normally run BPE on the individual bytes of UTF-8-encoded text. That is, we take a Unicode representation of text as a series of code points, encode it in bytes using UTF-8, and we treat each of these individual bytes as the input to BPE. Thus BPE likely begins by rediscovering the 2-byte and common 3-byte sequences that UTF-8 uses to encode various code points. Again, running BPE only inside presegmented words helps avoid problems. Because there are only 256 possible values of a byte, there will be no unknown tokens, although it's possible that BPE will learn some illegal UTF-8 sequences across character boundaries. These will be very rare, and can be eliminated with a filter.

Let's see some examples of the industrial application of the BPE tokenizer used in large systems like OpenAI GPT-4o. This tokenizer has 200K tokens, which is a comparatively large number. We can use Tat Dat Duong's Tiktokenizer visualizer

(https://tiktokenizer.vercel.app/) to see the number of tokens in a given sentence. For example here's the tokenization of a nonsense sentence we made up; the visualizer uses a center dot to indicate a space:

```
Anyhow,·she's·seen·Jane's·224123·flowers·anyhow!
```

The visualization shows colors to separate out words, but of course the true output of the tokenizer is simply a sequence of unique token ids. (In case you're interested, they were the following 13 tokens: 11865, 8923, 11, 31211, 6177, 23919, 885, 220, 19427, 7633, 18887, 147065, 0)

Notice that most words are their own token, usually including the leading space. Clitics like `'s` are segmented off when they appear on proper nouns like `Jane`, but are counted as part of a word for frequent words like `she's`. Numbers tend to be segmented into chunks of 3 digits. And some words (like *anyhow*) are segmented differently if they appear capitalized sentence-initially (two tokens, `Any` and `how`), than if they appear after a space, lower case (one token `anyhow`).

pretokenization

Some of these are related to preprocessing steps. As we mentioned briefly above, language models usually create their tokens in a **pretokenization** stage that first segments the input using regular expressions, for example breaking the input at spaces and punctuation, stripping off clitics, and breaking numbers into sets of digits. We'll see how to use regular expressions in Section 2.6.

SuperBPE
BoundlessBPE

It's possible to change this pretokenization to allow BPE tokens to span multiple words. For example the **SuperBPE** (Liu et al., 2025) and **BoundlessBPE** (Schmidt et al., 2025) algorithms first induce regular BPE subword tokens by enforcing pretokenization. They then run a second stage of BPE allowing merges across spaces and punctuation. The result is a large set of tokens that can be more efficient (Fig. 2.7).

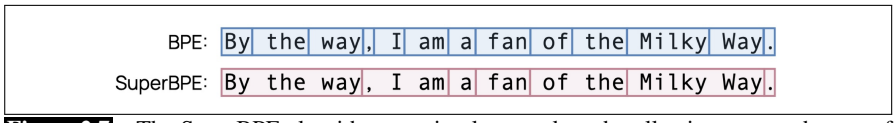


Figure 2.7 The SuperBPE algorithm creating larger tokens by allowing a second stage of merging across spaces. Figure from Liu et al. (2025).

Many of the tokenizers used in practice for large language models are multilingual, trained on many languages. But because the training data for large language models is vastly dominated by English text, these multilingual BPE tokenizers tend to use most of the tokens for English, leaving fewer of them for other languages. The result is that they do a better job of tokenizing English, and the other languages tend to get their words split up into shorter tokens. For example let's look at a Spanish sentence from a recipe for plantains, together with an English translation.

The English version has 19 tokens, with 13 of the 14 words as tokens, and only *nutmeg* split into 2 tokens (plus 3 commas and the final period).

```
In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·g
inger,·and·nutmeg.
```

By contrast, the original 16 words in Spanish have been encoded into 33 tokens, a much larger number. Notice that many basic words have been broken into pieces. For example `hondo`, 'deep', has been segmented into `h` and `ondo`. Similarly for `jugo`, 'juice', `nuez`, 'nut' and `jengibre` 'ginger'):

```
En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con
·el·azúcar,·jengibre,·y·nuez·moscada.
```

Spanish is not a low-resource language; this oversegmenting can be even more serious in lower resource languages, often down to individual characters. Oversegmenting into these tiny tokens can cause various problems for the downstream processing of the language. As will become more clear once we introduce transformer models in Chapter 7, such fragmentation can lead to poor representations of meaning, the need for longer contexts, and higher costs to train models (Rust et al., 2021; Ahia et al., 2023).

2.5 Corpora

Words don't appear out of nowhere. Any particular piece of text that we study is produced by one or more specific speakers or writers, in a specific dialect of a specific language, at a specific time, in a specific place, for a specific function.

Perhaps the most important dimension of variation is the language. NLP algorithms are most useful when they apply across many languages. The world has 7097 languages at the time of this writing, according to the online Ethnologue catalog (Simons and Fennig, 2018). It is important to test algorithms on more than one language, and particularly on languages with different properties; by contrast there is an unfortunate current tendency for NLP algorithms to be developed or tested just on English (Bender, 2019). Even when algorithms are developed beyond English, they tend to be developed for the official languages of large industrialized nations (Chinese, Spanish, Japanese, German etc.), but we don't want to limit tools to just these few languages. Furthermore, most languages also have multiple varieties, often spoken in different regions or by different social groups. Thus, for example, if we're processing text that uses features of African American English (**AAE**) or African American Vernacular English (AAVE)—the variations of English that can be used by millions of people in African American communities (King 2020)—we must use NLP tools that function with features of those varieties. Twitter posts might use features often used by speakers of African American English, such as constructions like *iont* (*I don't* in Mainstream American English (**MAE**)), or *talmbout* corresponding to MAE *talking about*, both examples that influence word segmentation (Blodgett et al. 2016, Jones 2015).

AAE

MAE

It's also quite common for speakers or writers to use multiple languages in a single utterance, a phenomenon called **code switching**. Code switching is enormously common across the world; here are examples showing Spanish and (transliterated) Hindi code switching with English (Solorio et al. 2014, Jurgens et al. 2017):

code switching

(2.12) Por primera vez veo a @username actually being hateful! it was beautiful:)
[For the first time I get to see @username actually being hateful! it was beautiful:)]

(2.13) dost tha or ra- hega ... dont wory ... but dherya rakhe
["he was and will remain a friend ... don't worry ... but have faith"]

We are focusing on text in this chapter (we'll turn to speech in Chapter 15), but it's important to note that many languages are primarily or exclusively oral. Speakers of these languages are often diglossic in a particular way, using an oral **vernacular** for family or local communication and another **vehicular** language, which is likely to have a written form, for participating in the larger economy or society or nation-state (Bird, 2020).

vernacular

vehicular

Another dimension of variation is the genre. The text that our algorithms must

The word "embedding" derives historically from its mathematical sense as a mapping from one space or structure to another, although the meaning has shifted; see the end of the chapter.

Fig. 5.1 shows a visualization of embeddings learned by the word2vec algorithm, showing the location of selected words (neighbors of "sweet") projected down from 200-dimensional space into a 2-dimensional space. Note that the nearest neighbors of *sweet* are semantically related words like *honey*, *candy*, *juice*, *chocolate*. This idea that similar words are neighbors in high-dimensional space offers enormous power to language models and other NLP applications. For example the sentiment classifiers of Chapter 4 depend on the same words appearing in the training and test sets. But by representing words as embeddings, a classifier can assign sentiment as long as it sees some words with *similar meanings*. And as we'll see, vector semantic models like the ones showed in Fig. 5.1 can be learned automatically from text without supervision.

In this chapter we'll begin with a simple pedagogical model of embeddings in which the meaning of a word is defined by a vector with the counts of nearby words. We introduce this model as a helpful way to understand the concept of vectors and what it means for a vector to be a representation of word meaning, but more sophisticated variants like the **tf-idf model** we will introduce in Chapter 11 are important methods you should understand. We will see that this method results in very long vectors that are **sparse**, i.e. mostly zeros (since most words simply never occur in the context of others). We'll then introduce the **word2vec** model family for constructing short, **dense** vectors that have even more useful semantic properties.

We'll also introduce the **cosine**, the standard way to use embeddings to compute *semantic similarity*, between two words, two sentences, or two documents, an important tool in practical applications.

5.3 Simple count-based embeddings

"The most important attributes of a vector in 3-space are {Location, Location, Location}"
Randall Munroe, the hover from https://xkcd.com/2358/

Let's now introduce the first way to compute word vector embeddings. This simplest vector model of meaning is based on the **co-occurrence matrix**, a way of representing how often words co-occur. We'll define a particular kind of co-occurrence matrix, the **word-context matrix**, in which each row in the matrix represents a word in the vocabulary and each column represents how often each other word in the vocabulary appears nearby. This matrix is thus of dimensionality $|V| \times |V|$ and each cell records the number of times the row (target) word and the column (context) word co-occur nearby in some training corpus.

word-context matrix

What do we mean by 'nearby'? We could implement various methods, but let's start with a very simple one: a context window around the word, let's say of 4 words to the left and 4 words to the right. If we do that, each cell will represent the number of times (in some training corpus) the column word occurs in such a ± 4-word window around the row word.

Let's see how this works for 4 words: *cherry*, *strawberry*, *digital*, and *information*. For each word we took a single instance from a corpus, and we show the ± 4 word window from that instance:

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

If we then take every occurrence of each word in a large corpus and count the context words around it, we get a word-context co-occurrence matrix. The full word-context co-occurrence matrix is very large, because for each word in the vocabulary (of size $|V|$) we have to count how often it occurs with every other word in the vocabulary, hence dimensionality $|V| \times |V|$. Let's therefore instead sketch the process on a smaller scale. Imagine that we are going to look at only the 4 words, and only consider the following 3 context words: *a*, *computer*, and *pie*. Furthermore let's assume we only count occurrences in the mini-corpus above.

So before looking at Fig. 5.2, compute by hand the counts for these 3 context words for the four words *cherry*, *strawberry*, *digital*, and *information*.

	a	**computer**	**pie**
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

Figure 5.2 Co-occurrence vectors for four words with counts from the 4 windows above, showing just 3 of the potential context word dimensions. The vector for *cherry* is outlined in red. Note that a real vector would have vastly more dimensions and thus be even sparser.

Hopefully your count matches what is shown in Fig. 5.2, so that each cell represents the number of times a particular word (defined by the row) occurs in a particular context (defined by the word column).

vector

Each row, then, is a vector representing a word. To review some basic linear algebra, a **vector** is, at heart, just a list or array of numbers. So *cherry* is represented as the list [1,0,1] (the first **row vector** in Fig. 5.2) and *information* is represented as the list [1,1,0] (the fourth row vector).

vector space
dimension

A **vector space** is a collection of vectors, and is characterized by its **dimension**. Vectors in a 3-dimensional vector space have an element for each dimension of the space. We will loosely refer to a vector in a 3-dimensional space as a 3-dimensional vector, with one element along each dimension. In the example in Fig. 5.2, we've chosen to make the word vectors of dimension 3, just so they fit on the page; in real word-word matrices, the vectors would have dimensionality $|V|$, the vocabulary size.

The ordering of the numbers in a vector space indicates the different dimensions on which words vary. The third dimension for all these vectors corresponds to the number of times *pie* occurs in the context. The second dimension for all of them corresponds to the number of times the word *computer* occurs. Notice that the vectors for *information* and *digital* have the same value (1) for this "computer" dimension.

In reality, we don't compute word vectors on a single context window. Instead, we compute them over an entire corpus. Let's see what some real counts look like. Let's look at some vectors computed in this way. Fig. 5.3 shows a subset of the word-word co-occurrence matrix for these four words, where, again because it's impossible to visualize all $|V|$ possible context words on the page of this textbook, we show a subset of 6 of the dimensions, with counts computed from the Wikipedia corpus (Davies, 2015).

Note in Fig. 5.3 that the two words *cherry* and *strawberry* are more similar to

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

Figure 5.3 Co-occurrence vectors for four words in the Wikipedia corpus, showing six of the dimensions (hand-picked for pedagogical purposes). The vector for *digital* is outlined in red. Note that a real vector would have vastly more dimensions and thus be much sparser, i.e. would have zero values in most dimensions.

each other (both *pie* and *sugar* tend to occur in their window) than they are to other words like *digital*; conversely, *digital* and *information* are more similar to each other than, say, to *strawberry*.

We can think of the vector for a word as a point in $|V|$-dimensional space; thus the words in Fig. 5.3 are points in 6-dimensional space. Fig. 5.4 shows a spatial visualization in 2 dimensions.

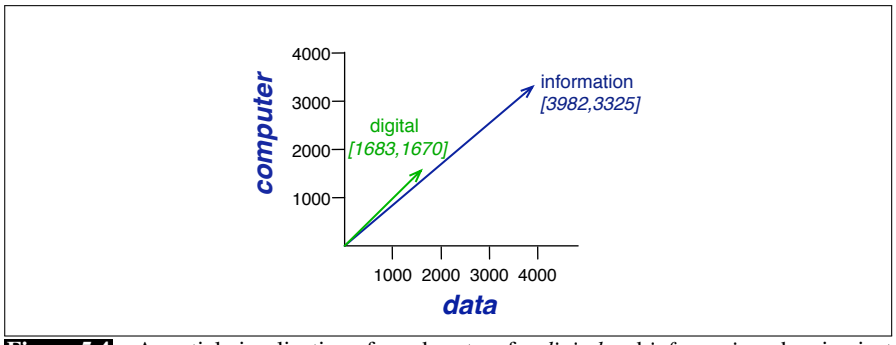


Figure 5.4 A spatial visualization of word vectors for *digital* and *information*, showing just two of the dimensions, corresponding to the words *data* and *computer*.

Note that $|V|$, the dimensionality of the vector, is generally the size of the vocabulary, often between 10,000 and 50,000 words (using the most frequent words in the training corpus; keeping words after about the most frequent 50,000 or so is generally not helpful). Since most of these numbers are zero these are **sparse** vector representations; there are efficient algorithms for storing and computing with sparse matrices.

It's also possible to apply various kinds of weighting functions to the counts in these cells. The most popular such weighting is tf-idf, which we'll introduce in Chapter 11, but there have historically been a wide variety of other weightings.

Now that we have some intuitions, let's move on to examine the details of computing word similarity.

5.4 Cosine for Measuring Similarity

To measure similarity between two target words v and w, we need a metric that takes two vectors (of the same dimensionality, either both with words as dimensions, hence of length $|V|$, or both with documents as dimensions, of length $|D|$) and gives a measure of their similarity. By far the most common similarity metric is the **cosine** of the angle between the vectors.

dot product
inner product

The cosine—like most measures for vector similarity used in NLP—is based on the **dot product** operator from linear algebra, also called the **inner product**:

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + ... + v_N w_N \tag{5.7}$$

The dot product acts as a similarity metric because it will tend to be high just when the two vectors have large values in the same dimensions. Alternatively, vectors that have zeros in different dimensions—orthogonal vectors—will have a dot product of 0, representing their strong dissimilarity.

vector length

This raw dot product, however, has a problem as a similarity metric: it favors **long** vectors. The **vector length** is defined as

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2} \tag{5.8}$$

The dot product is higher if a vector is longer, with higher values in each dimension. More frequent words have longer vectors, since they tend to co-occur with more words and have higher co-occurrence values with each of them. The raw dot product thus will be higher for frequent words. But this is a problem; we'd like a similarity metric that tells us how similar two words are regardless of their frequency.

We modify the dot product to normalize for the vector length by dividing the dot product by the lengths of each of the two vectors. This **normalized dot product** turns out to be the same as the cosine of the angle between the two vectors, following from the definition of the dot product between two vectors $\mathbf{a}$ and $\mathbf{b}$:

$$\begin{aligned} \mathbf{a}\cdot\mathbf{b} &= |\mathbf{a}||\mathbf{b}|\cos\theta \\ \frac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{a}||\mathbf{b}|} &= \cos\theta \end{aligned} \tag{5.9}$$

cosine

The **cosine** similarity metric between two vectors $\mathbf{v}$ and $\mathbf{w}$ thus can be computed as:

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}} \tag{5.10}$$

unit vector

For some applications we pre-normalize each vector, by dividing it by its length, creating a **unit vector** of length 1. Thus we could compute a unit vector from $\mathbf{a}$ by dividing it by $|\mathbf{a}|$. For unit vectors, the dot product is the same as the cosine.

The cosine value ranges from 1 for vectors pointing in the same direction, through 0 for orthogonal vectors, to -1 for vectors pointing in opposite directions. But since raw frequency values are non-negative, the cosine for these vectors ranges from 0–1.

Let's see how the cosine computes which of the words *cherry* or *digital* is closer in meaning to *information*, just using raw counts from the following shortened table:

	pie	**data**	**computer**
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325